Pokročilé objektové technológie

6

KST Katedra softvérových technológií
Fakulta riadenia a informatiky, Žilinská univerzita v Žiline

Komunikácia po sieti

Socket, Webové služby, WCF, REST, gRPC, Websocket

Úrovne sieťovej komunikácie

- Transportné protokoly
  - TCP
  - UDP
- Aplikačné protokoly
  - HTTP
- Technológie
  - Socket
  - WebSocket
  - REST
  - gRPC
  - SOAP (Webové služby)

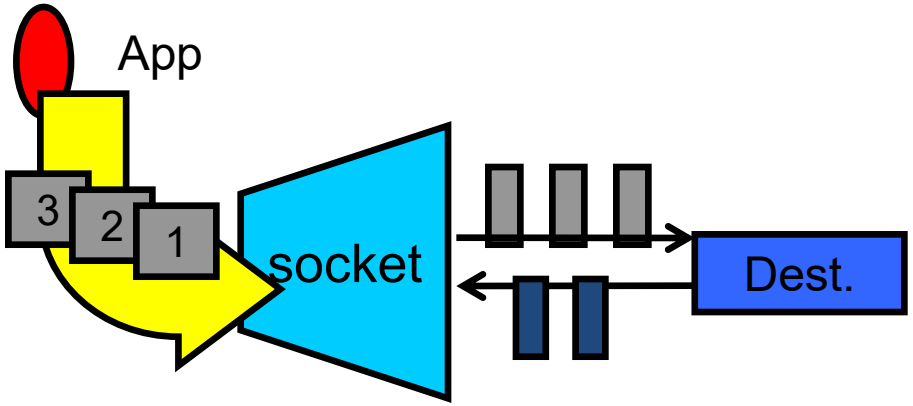
Čo je Socket

- Interface medzi aplikáciou a sieťou
  - Aplikácia vytvára socket
  - Typ socketu definuje spôsob komunikácie
    - connection-oriented resp. connectionless
- Po vytvorení a nakonfigurovaní, aplikácia môže
  - Posielať dáta do socketu na prenos sieťou
  - Prijímať dáta zo siete poslané iným host-om (aplikáciou)

Typy socket-ov

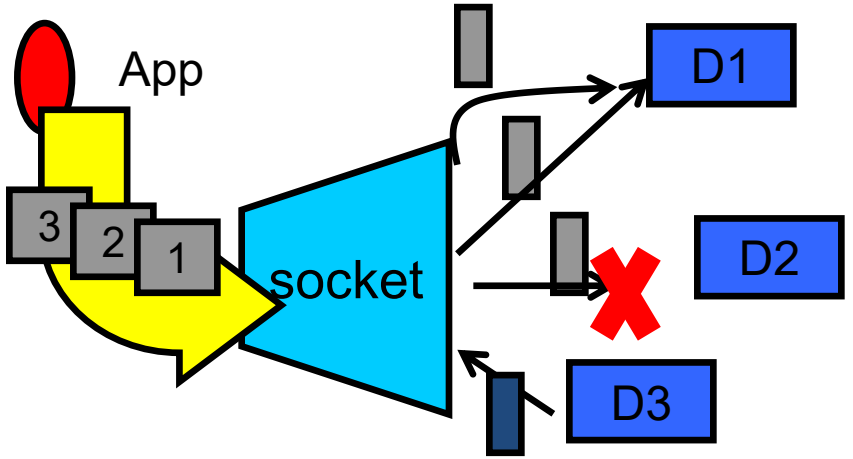
SOCK_STREAM

- V TCP/IP je to TCP
- Spoľahlivé doručenie
- Garancia poradia packet-ov
- Connection-oriented
- Obojstranný kanál



SOCK_DGRAM

- V TCP/IP je to UDP
- Nespoľahlivé doručenie
- Negarantované poradie
- Neexistuje spojenie – každý packet môže mať iného adresáta
- Môže prijímať aj vysielať



Na čo dva typy

SOCK_STREAM

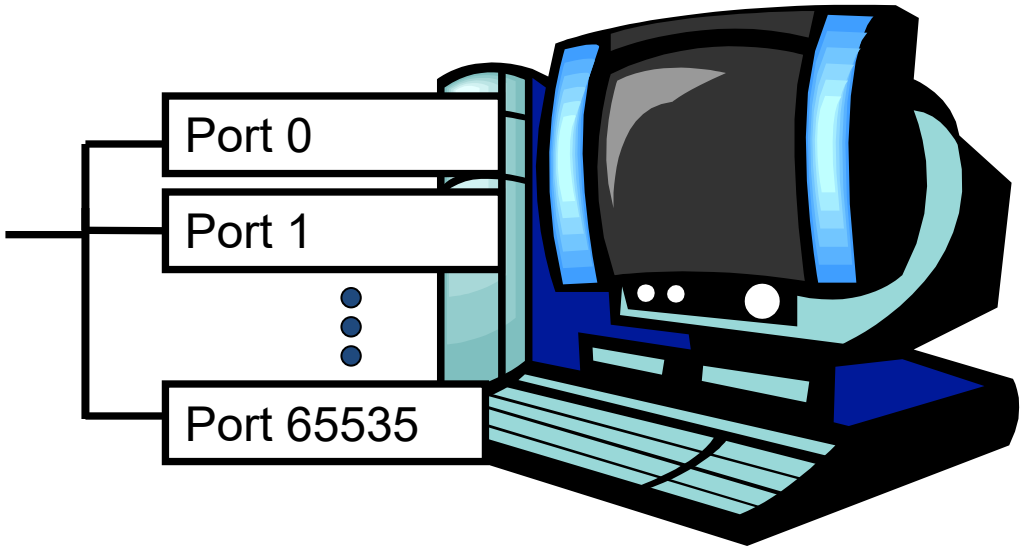
- HTTP, FTP, SMTP, POP3, IMAP, LDAP, Telnet
- HTTPS, SFTP, SMTPS, POP3S, IMAPS, LDAPS, SSH, ...

SOCK_DGRAM

- RTP, RTSP, RTMFP, NFS (aj), VPN, ...
- Vo všeobecnosti multimediálne protokoly – hlavne multicast-ové

Port

- Každý host má 65535 (2^{16})
- Niektoré porty sú rezervované
  - 20,21: FTP
  - 23: Telnet
  - 80: HTTP
  - RFC 1700 (cca 2000 portov je rezervovaných)



Bind

- Priradenie (môže byť exkluzívne) portu pre socket
- Nie vždy je potrebné vykonať bind
  - SOCK_DGRAM
    - Ak potrebujeme iba posielať dáta – OS nájde voľný port pri každom posielanom pakete
    - Pre príjem potrebujeme bind
  - SOCK_STREAM
    - Host server ku ktorému sa pripájame musí volať bind (definuje pevný port)
    - Host klient nepotrebuje volať bind – OS pridelí pri vytvorení spojenia

```
var listener = new TcpListener(IPAddress.Any, 23);
listener.Start();

while (true)
{
    using Socket socket = await listener.AcceptSocketAsync();
    int num;
    var buffer = new byte[10];
    Console.WriteLine("Connect from {0}", socket.RemoteEndPoint);
    await socket.SendAsync(Encoding.Default.GetBytes("login: "));
    bool firstPassword = true;
    while ((num = await socket.ReceiveAsync(buffer)) != 0)
    {
        string text = Encoding.Default.GetString(buffer, 0, num);
        Console.Write(text);
        if (text.Contains('\n'))
        {
            if (firstPassword) await socket.SendAsync(new byte[] { 0xff, 0xfd, 0x22 });
            else await socket.SendAsync(Encoding.Default.GetBytes("\r\nAccess denied\r\n"));
            await socket.SendAsync(Encoding.Default.GetBytes("Password: "));
            firstPassword = false;
        }
    }
    Console.WriteLine("Closed from {0}", socket.RemoteEndPoint);
}
```

Socket – použitie

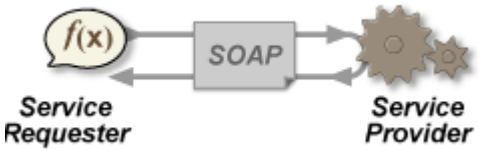
- Nevýhody
  - Všetko je nutné programovať
  - Neštandardné komunikačné protokoly
  - Často sa nepoužívajú binárne prenosy, ale textové
    - Big-Endian resp. Little-Endian
    - Zarovnávanie štruktúr – fragmentácia
- Výhody
  - Vývojár má všetko v rukách – vlastné komunikačné protokoly
  - Najnižšia úroveň – bez zbytočnej réžie

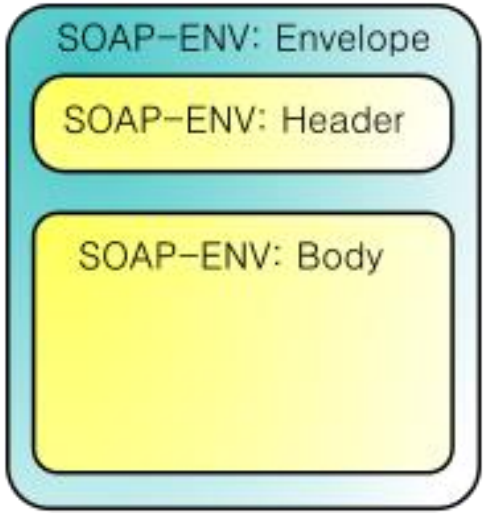
Webové služby

- Získavanie dát pomocou vzdialeného volania metód
- Používaný pri rôznych enterprise riešeniach
- Nezávislosť výmeny dát od platformy
- Možnosť vytvoriť klienta / servera v inom prog. jazyku
- Využívajú Simple Object Access Protocol (SOAP)
- Formát a operácie služby prístupné cez Web Service Description Language (WSDL)

SOAP

- Simple Object Access Protocol
- Založený na výmene XML správ
- Využíva zvyčajne HTTP
- Každá správa má presne definovanú štruktúru






```
POST /InStock HTTP/1.1
Host: www.example.org
Content-Type: application/soap+xml; charset=utf-8
Content-Length: 299
SOAPAction: "http://www.w3.org/2003/05/soap-envelope"

<?xml version="1.0"?>
<soap:Envelope xmlns:soap="http://www.w3.org/2003/05/soap-envelope" xmlns:m="http://www.example.org">
  <soap:Header>
  </soap:Header>
  <soap:Body>
    <m:GetStockPrice>
      <m:StockName>T</m:StockName>
    </m:GetStockPrice>
  </soap:Body>
</soap:Envelope>
```

https://docs.microsoft.com/en-us/dynamics365/business-central/dev-itpro/webservices/soap-web-services

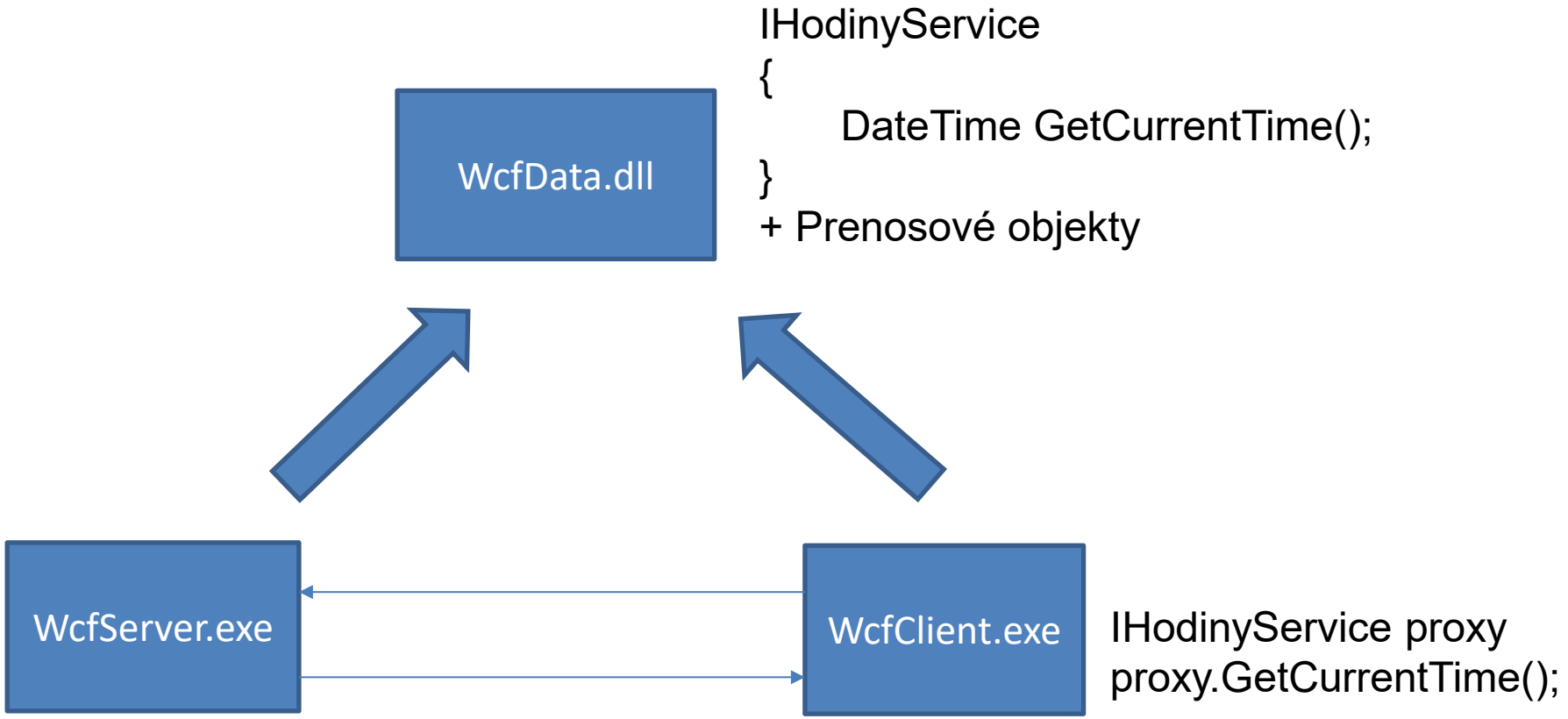
Windows Communication Foundation (WCF)

- Framework pre distribuované aplikácie
- Umožňuje tvorbu SOA aplikácií
- Implementuje WS-* štandardy
- Jednotné API pre:
  - .NET Remoting
  - ASP.NET Web Services
  - Enterprise Services (COM+)
  - Microsoft Message Queuing (MSMQ)

WCF

- Zjednodušenie vývoja
  - Jedno API pre viac komunikačných technológií
  - V budúcnosti môže obsahovať ďalšie
- Flexibilita
  - Zmena komunikačnej technológie neznamená zmenu kódu
  - Možnosť viacerých kom. technológií pre jednu službu
- Interoperabilita
  - Obsahuje štandardy (ne-Microsoft)
  - Schopnosť prepojenia aplikácií rôznych platforiem

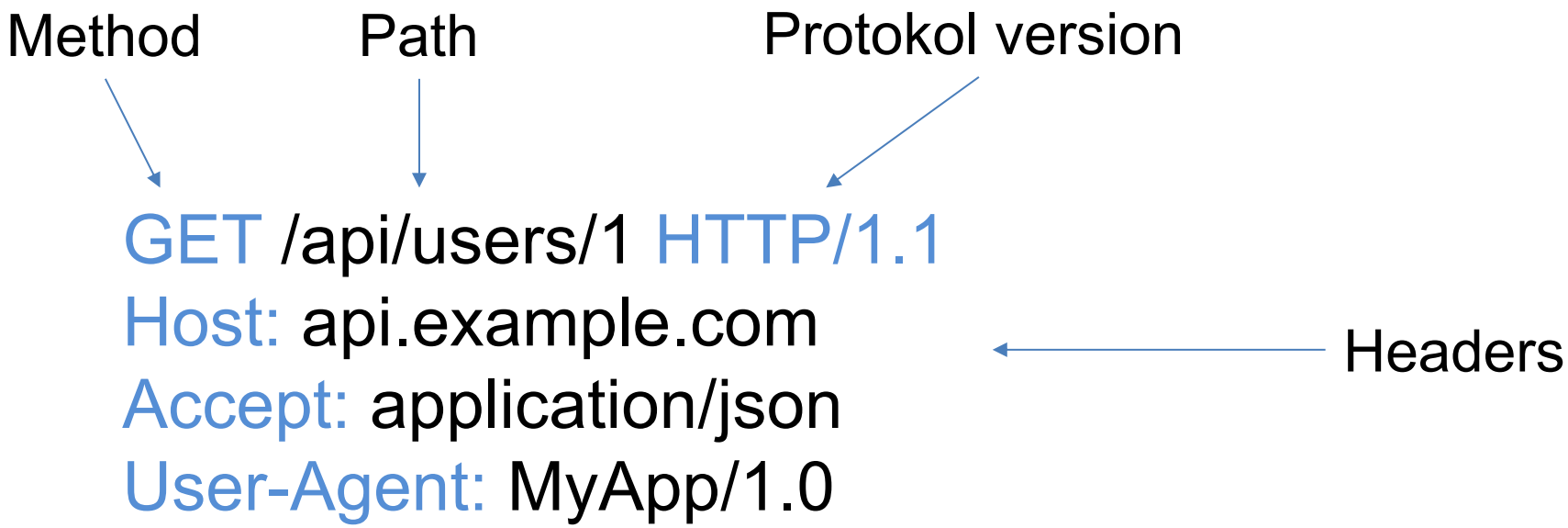
WCF – Architektúra aplikácie



Web API

- **Čo je Web API:**
  - Rozhranie, ktoré umožňuje aplikáciám **komunikovať cez web**
  - Najčastejšie používa **HTTP/HTTPS protokol**
  - Dáta sa prenášajú vo formáte **JSON** alebo **XML**
- **Ako funguje:**
  - Klient (napr. web, mobilná appka) → **HTTP request**
  - Server (API) → spracuje požiadavku a vráti **HTTP response**

HTTP Request



HTTP Response

Protokol version

Status code

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: 47
Date: Tue, 04 Nov 2025 14:23:10 GMT

{
  "id": 1,
  "name": "Anna",
  "email": "anna@example.com"
}
```

Headers

Body

REST

- ***Re**presentational **S**tate **T**ransfer*
- Architektonický štýl (architektúra rozhrania)
  - API spĺňajúce všetky náležitosti REST API sa nazýva RESTful API
- Autor: R. T. Fielding (dizertačná práca)
- Spôsob získavania dát z aplikácií tretích strán pomocou HTTP/HTTPS protokolu
  - Prepojenie rôznych aplikácií na úrovni API
- Definuje dátové zdroje (resources)
  - Napr. pre Single page aplikácie

Základné princípy RESTful webových služieb

- Používanie HTTP metód
- Služby musia byť bezstavové
- URI (URL) adresy vo forme adresárovej štruktúry
- Prenos dát vo forme JSON ale XML (alebo obidvoch)

Používanie HTTP metód

- GET – metóda na získavanie zdrojov
- POST – metóda na vytváranie zdrojov na serveri
- PUT – zmena stavu zdroja, alebo jeho úprava (nie všetky web servery ju podporujú)
- DELETE – vymazanie zdroja

- Zlý návrh:

~~GET /updateuser?name=Robert&newname=Bob HTTP/1.1~~

Dobrý návrh:

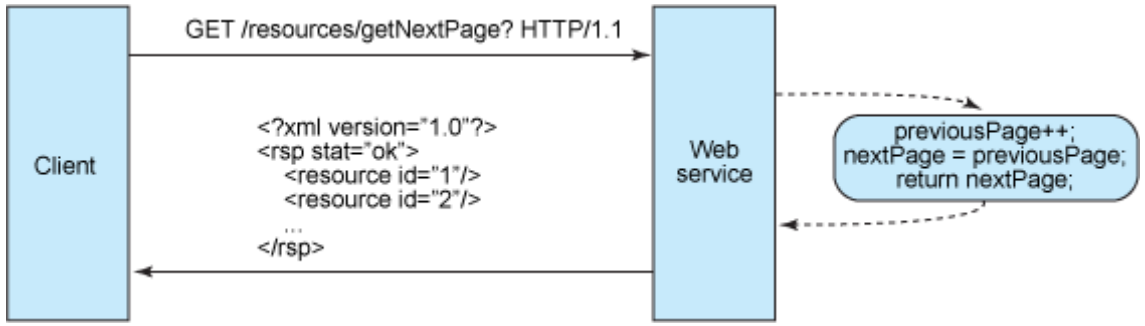
```
PUT /users/Robert HTTP/1.1
Host: myserver
Content-Type: application/json

{
  "name" : "Bob"
}
```

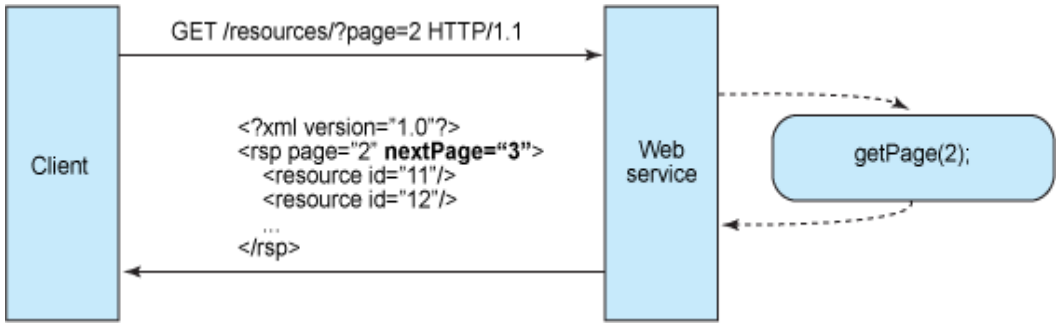
Bezstavovosť služieb

- Zaručujú menej komplikovaný vývoj aplikácie (viac výkonné)
- Posúvajú zodpovednosť za manažment stavu na klienta
- Jednotlivé žiadosti klienta sú na sebe nezávislé

Stavová aplikácia:



Bezstavová aplikácia:



URI adresárová štruktúra

- Spôsob zaručuje intuitívnosť používania webovej služby
  - Všetko malými písmenami
  - Medzery nahradiť pomlčkami, alebo podčiarkovníkmi (jedným, alebo druhým)
  - Nepoužívať URL parametre (časť za ?)
  - URL adresy by mali byť nemenné
- Príklad:

http://www.myservice.org/discussion/{year}/{day}/{month}/{topic} =>
http://www.myservice.org/discussion/2020/11/30/General+Info

Formáty prenosu dát

- Najčastejšie JSON
- Alternatívne XML
- Dobrý návrh API by mal poskytovať obidva formáty dát súčasne
  - Koncept: vyjednávanie o obsahu (content negotiation)
  - Klient si formát dát (JSON resp. XML) môže vyžiadať pomocou HTTP hlavičky:

    Accept: application/json

    resp.

    Accept: application/xml

Porovnanie webových a RESTful služieb

Webové služby

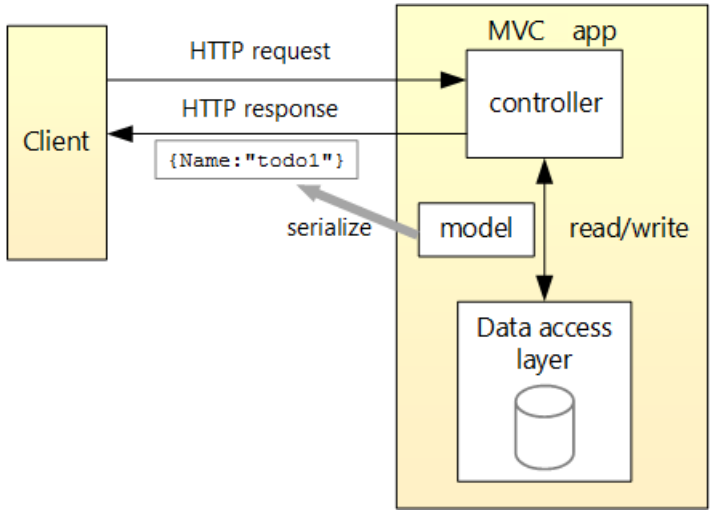
- Komplexné na implementáciu
- Určený najmä pre podnikové aplikácie
- Založené na SOAP (HTTP)
- Orientovaný na funkcie
- Formát dát XML
- Štandardizácia W3C
- Bezpečnosť: WS-Security, SSL
- Definovanie poskytovaných služieb cez WSDL
- Množstvo rozšírení

RESTful služby

- Jednoduché na implementáciu
- Určený pre verejné API (napr. sociálne siete)
- Založené na HTTP
- Orientovaný na dáta (zdroje)
- Rôzne formáty dát (XML, JSON, atď.)
- Architektonický štýl
- Bezpečnosť: SSL (HTTPS)
- Neobsahuje definíciu služieb (*Swagger*, *Postman*)

Tvorba REST rozhrania v .NET

- Založené na kontroleroch
  - 1 zdroj = 1 Controller
  - 1 operácia = 1 metóda
- Minimal API



- Automatické vyjednávanie o obsahu – štandardne JSON, XML
  - Možnosť špecifikovať formát
- URL adresy nastavujeme pomocou routovania – štandardne pomocou atribútov

ASP.NET core controller

```
[ApiController]
[Route("[controller]")]
public class WeatherForecastController : ControllerBase
{
    [HttpGet(Name = "GetWeatherForecast")]
    public IEnumerable<WeatherForecast> Get()
    {
        return Enumerable.Range(1, 5).Select(index => new WeatherForecast
            {
                Date = DateOnly.FromDateTime(DateTime.Now.AddDays(index)),
                TemperatureC = Random.Shared.Next(-20, 55),
                Summary = Summaries[Random.Shared.Next(Summaries.Length)]
            })
            .ToArray();
    }
}
```

Api Controller

- Trieda, dedí od *ControllerBase*
- Na routovanie požiadaviek využíva attribute router
  - Atribúty:
    - [Route("**[controller]**")]
    - [HttpGet(Name = "GetWeatherForecast")]
- Metódy vrátia dáta -> automatická konverzia do JSON / XML
- Automatický binding vstupných parametrov

Minimal API

- V .NET 6 možnosť definovať jednoduché API priamo v Program.cs

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.MapGet("/weatherforecast", () =>
{
    var forecast = Enumerable.Range(1, 5).Select(index =>
        new WeatherForecast
        (
            DateOnly.FromDateTime(DateTime.Now.AddDays(index)),
            Random.Shared.Next(-20, 55),
            summaries[Random.Shared.Next(summaries.Length)]
        ))
        .ToArray();
    return forecast;
})
.WithName("GetWeatherForecast");


app.Run();
```

Spracovanie vstupných parametrov

```
// 1) Route parameter: /greet/John
app.MapGet("/greet/{name}", (string name) =>
{
    return Results.Ok($"Ahoj, {name}!");
});

// 2) Query parameters: /add?x=3&y=4
app.MapGet("/add", (int x, int y) =>
{
    return Results.Ok(new { x, y, sum = x + y });
});

// 3) Optional query parameter with default: /hello or /hello?name=Eva
app.MapGet("/hello", (string? name) =>
{
    name ??= "neznámy";
    return $"Ahoj, {name}!";
});
```

Spracovanie vstupných parametrov 2

```
public record Person(string FirstName, string LastName, int Age);

// 4) POST with model binding from JSON body

app.MapPost("/person", ([FromBody] Person person) =>
{
    if (person is null) return Results.BadRequest("Chýba telo požiadavky (Person).");
    if (person.Age < 0) return Results.BadRequest("Vek musí byť nezáporné číslo.");

    // V reálnej aplikácii by ste uložili person do DB
    return Results.Created($"/person/{person.FirstName}", person);
});

// 5) Injected service (ILogger) and CancellationToken example
app.MapGet("/time", (ILogger<Program> logger, CancellationToken ct) =>
{
    logger.LogInformation("Požiadavka na /time prijatá.");
    // CancellationToken je už pripravený a bude signalizovaný pri zrušení požiadavky klientom
    return Results.Ok(new { utcNow = DateTime.UtcNow, cancelled = ct.IsCancellationRequested });
});
```

Asynchrónne operácie

```
// 6) Async endpoint that respects CancellationToken
app.MapGet("/wait", async (CancellationToken ct) =>
{
    // Simulujeme prácu, ktorú možno zrušiť
    await Task.Delay(5000, ct);
    return Results.Ok("Hotovo po čakaní 5s (ak to nebolo zrušené).");
});
```

Validácia dát

- Základná validácia

```
app.MapGet("/greet/{name}", (string name) =>
{
    if (string.IsNullOrWhiteSpace(name))
        return Results.BadRequest(new { error = "Meno nesmie byť prázdne." });

    if (name.Length > 50)
        return Results.BadRequest(new { error = "Meno je príliš dlhé (max 50
znakov)." });

    return Results.Ok($"Ahoj, {name}!");
});
```

Validácia pomocou DataAnnotations

- Definícia na triede

```
public record PersonValidated(
    [property: Required(ErrorMessage = "FirstName je povinné.")]
    [property: StringLength(100, MinimumLength = 1, ErrorMessage = "FirstName musí
mať 1-100 znakov.")]
    string FirstName,

    [property: Required(ErrorMessage = "LastName je povinné.")]
    [property: StringLength(100, MinimumLength = 1, ErrorMessage = "LastName musí
mať 1-100 znakov.")]
    string LastName,

    [property: Range(0, 150, ErrorMessage = "Age musí byť medzi 0 a 150.")]
    int Age
);
```

Validácia

```
// POST endpoint s validáciou DataAnnotations: /person2
app.MapPost("/person2", ([FromBody] PersonValidated? person) =>
{
    if (person is null)
        return Results.BadRequest(new { error = "Chýba telo požiadavky (Person)." });

    // Manual DataAnnotations validation
    var validationResults = new List<ValidationResult>();
    var ctx = new ValidationContext(person);
    if (!Validator.TryValidateObject(person, ctx, validationResults, validateAllProperties: true))
    {
        var errors = validationResults
            .SelectMany(r => (r.MemberNames?.Any() == true ? r.MemberNames : new[] { string.Empty })
                .Select(m => new { Member = m, r.ErrorMessage }))
            .GroupBy(x => x.Member)
            .ToDictionary(g => g.Key, g => g.Select(x => x.ErrorMessage).ToArray());

        return Results.ValidationProblem(errors);
    }

    // V reálnej aplikácii by ste uložili person do DB
    return Results.Created($"/person/{person.FirstName}", person);
});
```

Extrakcia validácie do Endpoint Filtra

- Definícia generického endpoint filtra
- Použitie na daný endpoint

```
app.MapPost("/person3", ([FromBody] Person? person) =>
    {
        // handler sa môže teraz sústrediť len na spracovanie platného modelu
        return Results.Created($"/person/{person!.FirstName}", person);
    })
    .AddEndpointFilter<ValidationFilter<Person>>(); // aplikovanie filtra
```

Popis REST API

- Štandard OPEN API
  - Formát JSON
  - Popis
    - Endpointov
    - Vstupných parametrov
    - Návratových hodnôt
    - Dátových štruktúr
  - Automatické generovanie kódu

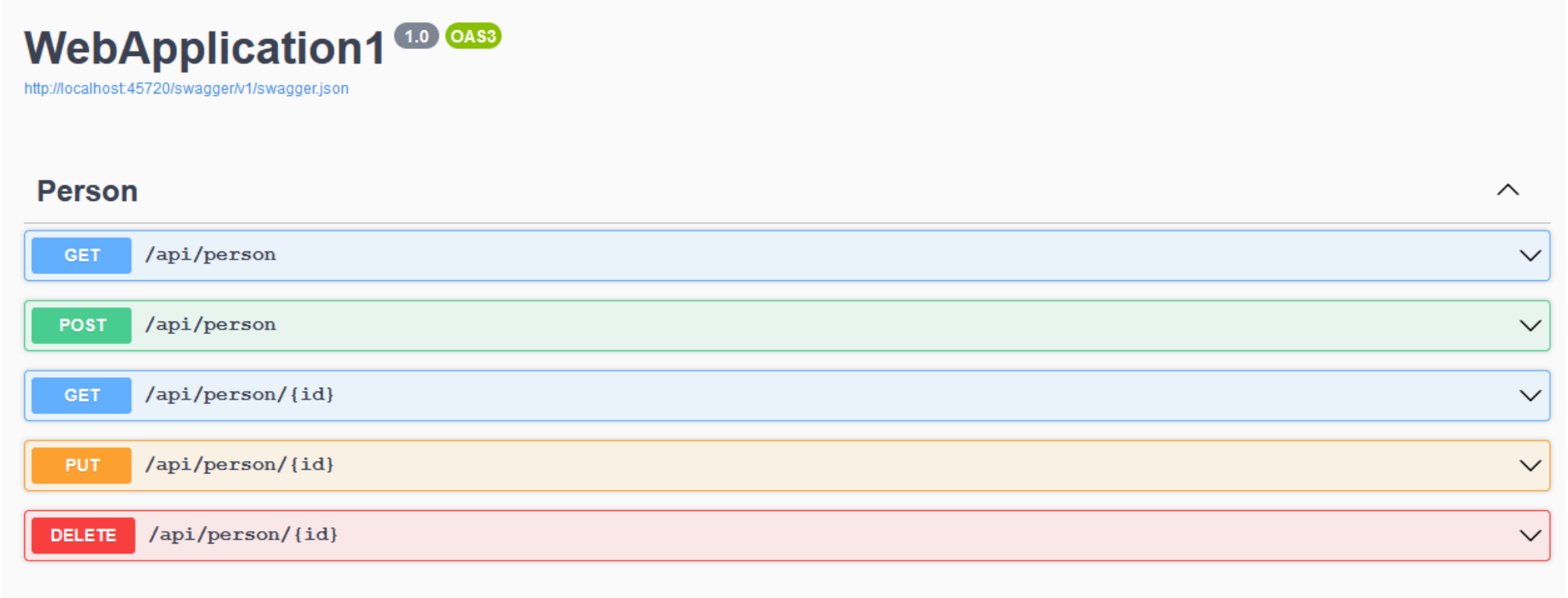
JSON

```
{
  "openapi": "3.0.1",
  "info": {
    "title": "API V1",
    "version": "v1"
  },
  "paths": {
    "/api/Todo": {
      "get": {
        "tags": [
          "Todo"
        ],
        "operationId": "ApiTodoGet",
        "responses": {
          "200": {
            "description": "Success",
            "content": {
              "text/plain": {
                "schema": {
                  "type": "array",
                  "items": {
                    "$ref": "#/components/schemas/ToDoItem"
                  }
                }
              },
```

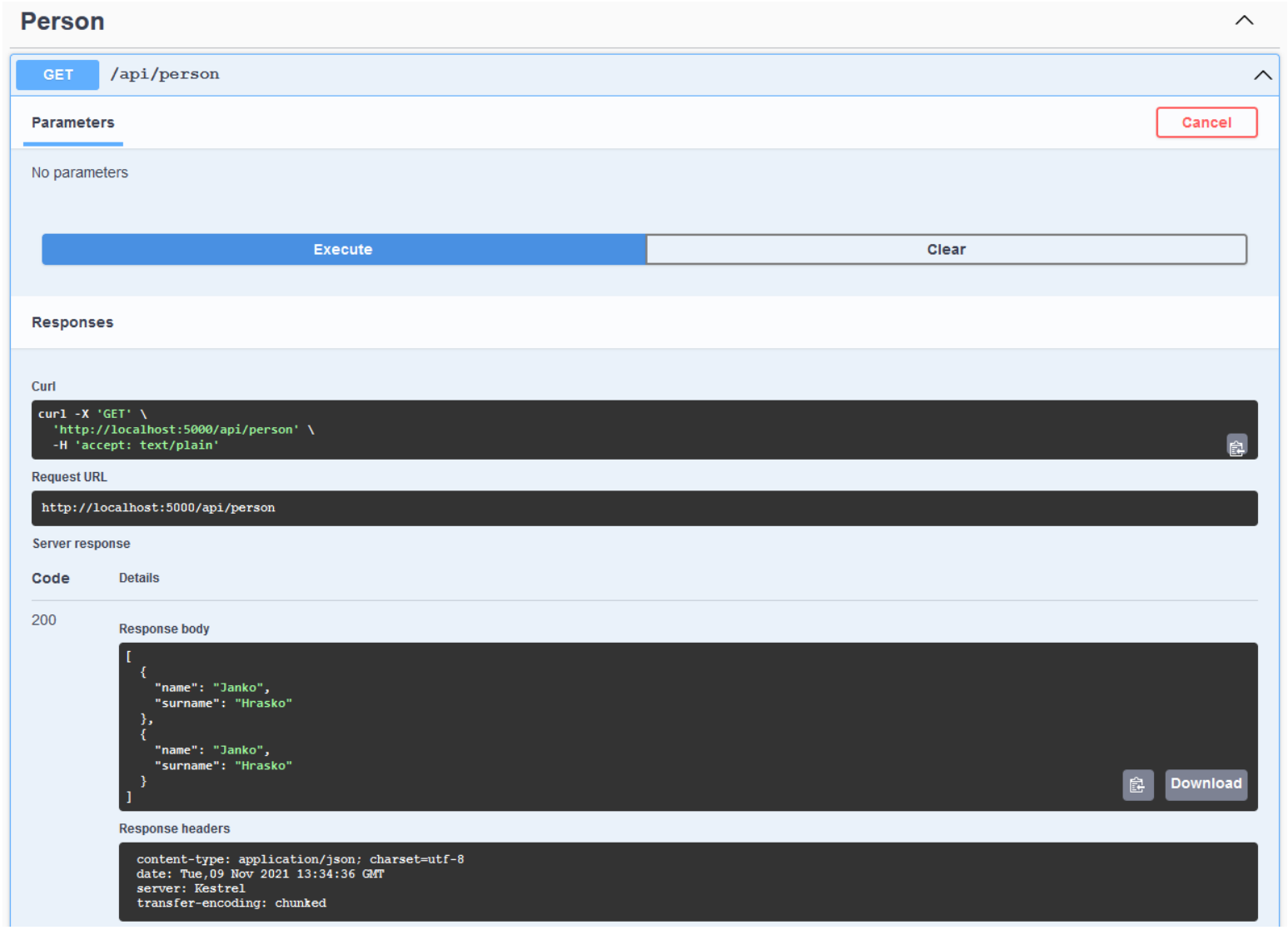
```
"components": {
  "schemas": {
    "ToDoItem": {
      "type": "object",
      "properties": {
        "id": {
          "type": "integer",
          "format": "int32"
        },
        "name": {
          "type": "string",
          "nullable": true
        },
        "isCompleted": {
          "type": "boolean"
        }
      },
      "additionalProperties": false
    }
  }
}
```

Swagger

- Predchodca OpenAPI
- Nástroj na automatické generovanie API dokumentácie

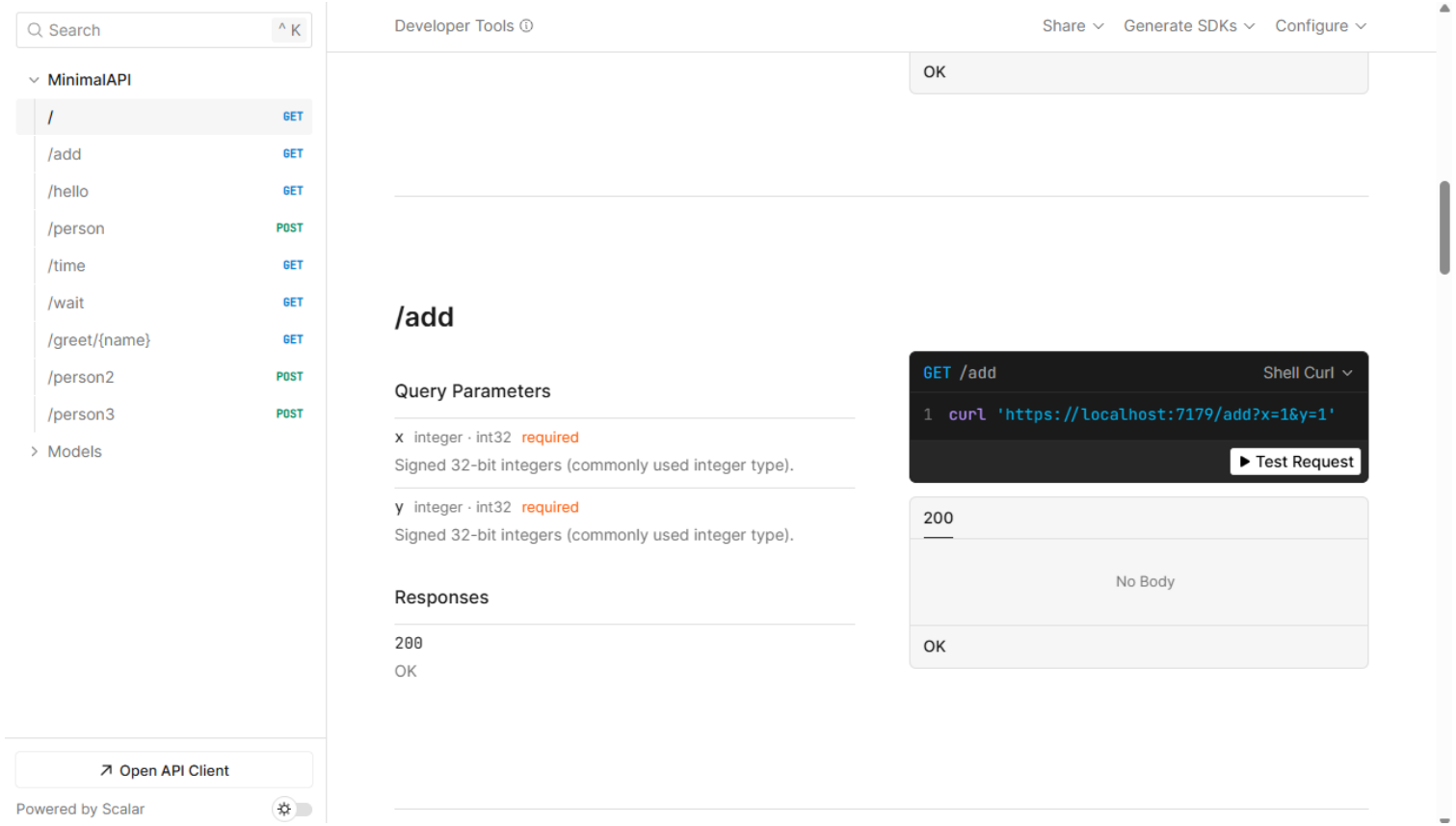


Swagger – testovanie API



Scalar

- Balíček Scalar.AspNetCore
- Alternatíva k swagger



KST Katedra softvérových technológií
Fakulta riadenia a informatiky, Žilinská univerzita v Žiline

Pripojenie na REST službu

- Základom je trieda HttpClient
- Rozširujúci balíček *Microsoft.Extensions.Http*

```
0 references
static async Task Main(string[] args)
{
    var client = new HttpClient();
    var persons = await client.GetFromJsonAsync<List<Person>>( requestUri: "http://localhost:5000/api/person");
    var response = await client.PostAsJsonAsync( requestUri: "http://localhost:5000/api/person",
        value: new Person( Name: "Michal",  Surname: "Duracik")); // Task<HttpResponseMessage>
}
```


KST Katedra softvérových technológií
Fakulta riadenia a informatiky, Žilinská univerzita v Žiline

Prenosové entity medzi klientom a serverom

- Možnosť definovať vlastné
- Zdieľaná knižnica
- Automatické vygenerovanie na základe OpenAPI
  - NSwag.CodeGeneration.Csharp
  - NSwagStudio
  - Scalar

REST - Zhrnutie

- Založené na protokole HTTP
  - Jednosmerná komunikácia
  - Klient žiada dáta
  - Server na žiadosť poskytuje dáta

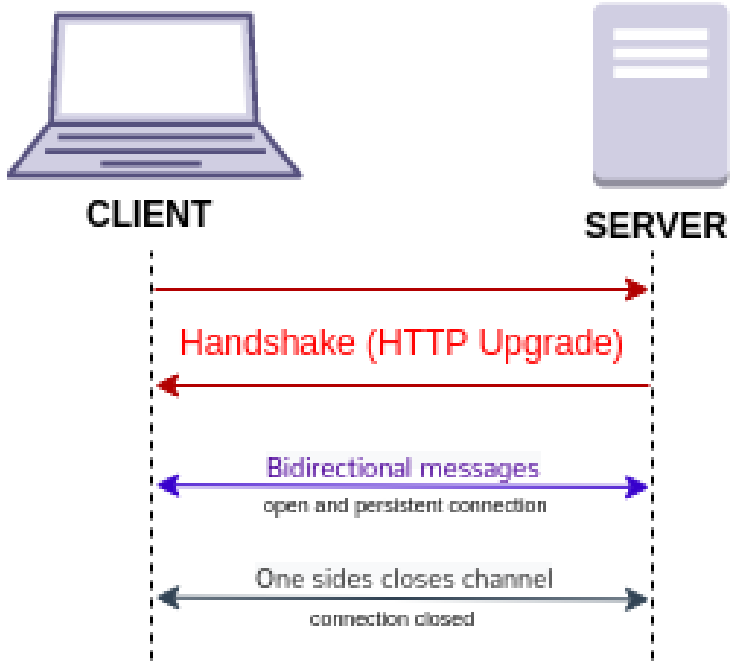
- Dáta sa generujú vtedy, keď ich potrebuje klient
  - Server nemá možnosť poslať dáta klientovi keď vzniknú

Web komunikácia v reálnom čase

- Websocket
- Long pooling
  - Pozdržanie odpovede, kým nie sú k dispozícii dáta
- Server Sent events
  - Kontinuálne streamovanie dát zo servera
- Forever Frame
  - Kontinuálne streamovanie dát zo servera (starý IE)

Websocket

- Komunikačný protokol nad HTTP
- Obojsmerná komunikácia
- Multiplatformový štandard
- Štandardne používa JSON správy
- Možnosť prenosu textových aj binárnych správ



Websocket v ASP.NET

- Pridanie middleware
  - app.UseWebSockets();
- Spracovanie pripojenia

```
public class WebSocketController : ControllerBase
{
    [HttpGet("/ws")]
    public async Task Get()
    {
        if (HttpContext.WebSockets.IsWebSocketRequest)
        {
            using WebSocket webSocket = await
                                HttpContext.WebSockets.AcceptWebSocketAsync();
            await Echo(HttpContext, webSocket);
        }
        else
        {
            HttpContext.Response.StatusCode = (int)HttpStatusCode.BadRequest;
        }
    }
```

https://docs.microsoft.com/en-us/aspnet/core/fundamentals/websockets

Websocket v ASP.NET

- Obsluha websocketu

```
private async Task Echo(HttpContext context, WebSocket webSocket)
{
    var buffer = new byte[1024 * 4];
    WebSocketReceiveResult result = await webSocket.ReceiveAsync(new ArraySegment<byte>(buffer), CancellationToken.None);
    while (!result.CloseStatus.HasValue)
    {
        await webSocket.SendAsync(new ArraySegment<byte>(buffer, 0, result.Count), result.MessageType, result.EndOfMessage, CancellationToken.None);

        result = await webSocket.ReceiveAsync(new ArraySegment<byte>(buffer), CancellationToken.None);
    }
    await webSocket.CloseAsync(result.CloseStatus.Value, result.CloseStatusDescription, CancellationToken.None);
}
```

https://docs.microsoft.com/en-us/aspnet/core/fundamentals/websockets

Pripojenie na websocket z .NET klienta

- Trieda ClientWebSocket
- Metódy
  - ConnectAsync()
  - ReceiveAsync()
  - SendAsync

SignalR

- Knižnica na Real time web komunikáciu
- API na tvorbu server-to-client RPC
- Vykonávanie funkcií na klientovi zo servera a naopak

- Automatická správa spojenia
- Preposielanie správ všetkých pripojeným klientom naraz
- Posielanie správ konkrétnym klientom / skupinám klientov

SignalR – podpora

- Server – ASP.NET
- Klient
  - .NET
  - Javascript
  - Java
  - Python
  - ...

SignalR - Transport

- SignalR na pozadí využíva jeden z nasledujúcich spôsobov komunikácie
  - WebSockety
  - Server-Sent Events
  - Long Polling
  - Forever Frame

SignalR - Hub

- Endpoint, ktorý:
  - Poskytuje API na volanie metód na pripojených klientoch
  - Definuje metódy, ktoré môže volať klient
- Bezstavový – pre každý request nová inštancia (transient)
- Po skončení requestu zaniká
- Pri asynchrónnych requestoch vždy používať await

SignalR – Stavové informácie

- *Context* property
  - ConnectionId
  - User
  - Items
  - Features
  - Abort()
  - HttpContext
  - …

SignalR – Prístup ku pripojeným klientom

- *Clients* property
  - *All* – umožňuje volať metódu na všetkých pripojených klientoch
  - *Current* – volanie metód na aktuálnom klientovi
  - *Other* – volanie metód na ostatných klientov

  - Možnosť spájať klientov podľa:
    - User id
    - Skupín

SignalR – Príklad ChatHub

```
public class ChatHub : Hub
{
    0 references
    public Task SendMessage(string user, string message)
    {
        return Clients.All.SendAsync( method: "ReceiveMessage",  arg1: user,  arg2: message);
    }
}
```

Startup.cs

```
app.UseEndpoints(endpoints :IEndpointRouteBuilder =>
{
    endpoints.MapGet( pattern: "/",  requestDelegate: async context =>
    {
        await context.Response.WriteAsync( text: "Hello World!");
    });
    endpoints.MapHub<ChatHub>( pattern: "/chathub");

});
```

SignalR – Odosielanie správy mimo Hubu

- IHubContext<T>
  - Property *Clients*
- Získanie pomocou DI v
  - Controlleroch
  - Razor pages
  - Hosted services

SignalR - .NET klient

- Microsoft.AspNetCore.SignalR.Client
- Pripojenie

```
var connection = new HubConnectionBuilder()
    .WithUrl("http://localhost:5000/chathub")
    .Build();
```

- Klientska metóda

```
connection.On<string, string>( methodName: "ReceiveMessage",  handler: (user :string, message :string) =>
{
    Console.WriteLine($"{user}: {message}");
});
```

- Odosielanie správy

```
await connection.SendAsync( methodName: "SendMessage",  arg1: username,  arg2: message);
```

SignalR – Typová kontrola remote metód

- Hub<T>, kde T je interface popisujúci správy na klientoch

```
public interface IChatClient
{
    2 references
    Task ReceiveMessage(string user, string message);
}


public class StronglyTypedChatHub : Hub<IChatClient>
{
    0 references
    public Task SendMessage(string user, string message)
    {
        return Clients.All.ReceiveMessage(user, message);
    }
}
```

SignalR – Serializácia

- Štandardne JSON
- Možnosť použiť MessagePack – binárna serializácia

https://docs.microsoft.com/en-us/aspnet/core/signalr/messagepackhubprotocol

gRPC

- Framework na vysokovýkonné RPC
- Multiplatformový (dostupnosť pre rôzne prog. jazyky)
- Contract-first API development
- Využíva Protocol Buffers
- Obojsmerný prenos dát
- Postavený nad protokolom HTTP/2

gRPC – Definícia kontraktu

- **.proto* súbory
- Definícia
  - Metód
  - Správ

```
syntax = "proto3";

service Greeter {
  rpc SayHello (HelloRequest) returns (HelloReply);
}

message HelloRequest {
  string name = 1;
}

message HelloReply {
  string message = 1;
}
```

*.proto syntax – správy

```
/*
Komentár
*/
message NázovSprávy {
  typParametra názov = poradie; //Komentár
}

message SearchRequest {
  string query = 1;
  int32 page_number = 2;
  int32 result_per_page = 3;
}
```

*.proto syntax – poradie parametrov

- Každý člen ma unikátne poradie
- Nesmie sa meniť (spätná kompatibilita)
- Niektoré môžete rezervovať „do budúcnosti“

*.proto syntax – typy

Základné

.proto	.NET
double	double
float	float
int32	int
int64	long
uint32	uint
uint64	ulong
bool	bool
string	string
bytes	ByteString

„Objektové“

```
message SearchResponse {
  repeated Result results = 1;
}

message Result {
  string url = 1;
  string title = 2;
  repeated string snippets = 3;
}
```

*.proto syntax – import iných typov

- Import pomocou
  - import "myproject/other_protos.proto";
- Možnosť importovať ďalšie bežne používané typy
  - import "google/protobuf/duration.proto";
    - //TimeSpan - google.protobuf.Duration
  - import "google/protobuf/timestamp.proto";
    - //DateTime - google.protobuf.Timestamp
- Nullable typy
  - google.protobuf.Int32Value ...

*.proto syntax – opakovanie prvkov (kolekcia)

- Kľúčové slovo **repeated**

```
message SearchResponse {
  repeated Result results = 1;
}

message Result {
  string url = 1;
  string title = 2;
  repeated string snippets = 3;
}
```

- Dictionary
  - map<string, Result > results = 3;

*.proto syntax - služby

```
/*
Komentár
*/
service NázovSlužby {
  rpc NázovMetód (TypVstupnehoParametra) returns (TypOdpovede);
//Komentár
}
service FooService {
  rpc GetFoo (GetFooRequest) returns (GetFooResponse);
  rpc CreateOrUpdateFoo (CreateOrUpdateFooRequest)
       returns (CreateOrUpdateFooResponse);
}
```

*.proto syntax – stream dát

```
service ExampleService {
  // Unary
  rpc UnaryCall (ExampleRequest) returns (ExampleResponse);

  // Server streaming
  rpc StreamingFromServer (ExampleRequest) returns (stream ExampleResponse);

  // Client streaming
  rpc StreamingFromClient (stream ExampleRequest) returns (ExampleResponse);

  // Bi-directional streaming
  rpc StreamingBothWays (stream ExampleRequest) returns (stream
ExampleResponse);
}
```

gRPC v .NET

- Nutnosť definovať .proto súbor
- Automatické vygenerovanie štruktúr a služieb na serveri
- Automatické vygenerovanie klienta

gRPC Service

```
public class GreeterService : Greeter.GreeterBase
{
    private readonly ILogger<GreeterService> _logger;
    0 references
    public GreeterService(ILogger<GreeterService> logger)
    {
        _logger = logger;
    }

    3 references
    public override Task<HelloReply> SayHello(HelloRequest request, ServerCallContext context)
    {
        return Task.FromResult(new HelloReply
        {
            Message = "Hello " + request.Name
        });
    }
```

gRPC .NET Client

```
// The port number must match the port of the gRPC server.
using var channel = GrpcChannel.ForAddress("https://localhost:5001");
var client = new Greeter.GreeterClient(channel);

var response :HelloReply = client.SayHello(new HelloRequest { Name = "test" });
Console.WriteLine(response.Message);
```

gRPC Web

- Webové prehliadače plne nepodporujú HTTP/2
- Možnosť postaviť HTTP proxy pre web aplikácie
- Grpc.AspNetCore.Web
- app.UseGrpcWeb();
- endpoints.MapGrpcService<GreeterService>().EnableGrpcWeb();

- Klient
- Grpc.Net.Client.Web

gRPC Web

- Pozor na CORS
- Nemá plné možnosti gRPC
  - Stream je možný len zo servera na klienta